

Guide Technique V1 — Pipeline Batch

Carnet Mental : Transaction Data Platform

1. Vue d'ensemble

1.1 Objectif du projet

Construire un pipeline de données qui transforme un fichier CSV brut de 6.36 millions de transactions en données exploitables dans PostgreSQL.

1.2 Pourquoi un pipeline batch ?

Le **batch** (lot) signifie qu'on traite les données par groupes, pas en temps réel. C'est le 模式 le plus simple et le plus courant pour démarrer.

MODE BATCH

Fichiers CSV → Traitement → PostgreSQL
(figés dans le temps)

Avantage : simple, debugable, reproductible
Inconvénient : pas en temps réel

1.3 Les 3 étapes fondamentales

ÉTAPE 1
EXTRACT
(Lire)

ÉTAPE 2
TRANSFORM
(Transformer)

ÉTAPE 3
LOAD
(Charger)

CSV source → Nettoyage
Timestamps
Validation → PostgreSQL

Règle d'or : On ne transforme JAMAIS les données brutes en place. Toujours : Brutes → Staging → Transformées.

2. Architecture technique

2.1 Stack technologique

Composant	Outil	Rôle
Langage	Python 3.12	Scripting, manipulation données
Données	pandas	Lecture/écriture CSV, transformations
Base	PostgreSQL 16	Stockage, requêtes
Connexion	SQLAlchemy 2.0	Pool de connexions, abstraction DB

Composant	Outil	Rôle
Driver	psycopg2-binary	Communication Python ↔ PostgreSQL
Conteneur	Docker	PostgreSQL isolé
Environnement	venv	Dépendances isolées

2.2 Structure du projet

```

Transaction_numerique/
├── data/
│   ├── source/      ← Fichier original (471 Mo)
│   ├── raw/         ← Données brutes (31 fichiers)
│   ├── processed/   ← Données transformées (31 fichiers)
│   └── sample/      ← Échantillon test (100 lignes)
├── src/
│   ├── ingestion/
│   │   └── ingest_batch.py      ← Étape 1 : Split CSV
│   ├── transformation/
│   │   └── transform_transactions.py ← Étape 2 : Transform
│   └── utils/
│       └── database.py         ← Étape 3 : Load PostgreSQL
├── sql/
│   └── create_tables.sql      ← Schéma de la table
├── tests/                    ← Tests unitaires
├── docker-compose.yml        ← PostgreSQL Docker
├── .env                      ← Credentials (non versionné)
└── requirements.txt          ← Dépendances Python

```

3. ÉTAPE 1 — Ingestion Batch

3.1 Concept

Le fichier source fait 6.36M de lignes. On le découpe en 31 fichiers journaliers (chaque fichier = 24 heures de données = 24 "steps").

3.2 Le concept de "step"

```

step = unité de temps (1 step = 1 heure)

Step 1  = 2024-01-01 00:00:00
Step 2  = 2024-01-01 01:00:00
...
Step 24 = 2024-01-01 23:00:00
Step 25 = 2024-01-02 00:00:00
...
Step 48 = 2024-01-02 23:00:00
...
Step 743 = 2024-01-31 22:00:00 (dernier step)

```

3.3 Code expliqué

```
# ingest_batch.py – Fonctions clés

# 1. Calcul du nombre de jours
def calculate_days(step_min, step_max):
    total_steps = step_max - step_min + 1      # 743 - 1 + 1 = 743
    num_days = (total_steps + 24 - 1) // 24    # (743 + 23) // 24 = 31
    return num_days

# 2. Découpage par jour
for day_num in range(1, num_days + 1):
    day_start = step_min + (day_num - 1) * 24  # Step début du jour
    day_end = min(day_start + 24 - 1, step_max) # Step fin du jour

    # Filtrer les lignes pour ce jour
    day_data = df[(df["step"] >= day_start) & (df["step"] <= day_end)]

    # Sauvegarder
    day_data.to_csv(f"day_{day_num:02d}.csv", index=False)
```

3.4 Résultat

```
Entrée : PS_20174392719_1491204439457_log.csv (6.36M lignes)
Sortie : day_01.csv (574K lignes)
        day_02.csv (455K lignes)
        day_03.csv (1K lignes)
        ...
        day_31.csv (272 lignes)
```

4. ÉTAPE 2 — Transformation

4.1 Concept

Les données brutes ont un champ `step` (entier). On le convertit en timestamp réel, on ajoute des colonnes dérivées, on nettoie, on valide.

4.2 Conversions

```
# Conversion step → timestamp
REFERENCE_DATE = datetime(2024, 1, 1, 0, 0, 0)

def convert_step_to_timestamp(step_value):
    return REFERENCE_DATE + timedelta(hours=int(step_value) - 1)

# Exemples :
# step 1 → 2024-01-01 00:00:00
# step 2 → 2024-01-01 01:00:00
# step 24 → 2024-01-01 23:00:00
# step 25 → 2024-01-02 00:00:00
```

4.3 Colonnes ajoutées

Colonne	Type	Exemple	Comment
transaction_date	TIMESTAMP	2024-01-01 00:00:00	Converti depuis step
transaction_date_only	DATE	2024-01-01	Partie date uniquement

Colonne	Type	Exemple	Comment
transaction_hour	INTEGER	0	Heure de la journée (0-23)

4.4 Nettoyage

```
# Suppression des doublons
df = df.drop_duplicates()

# Validation des types
df["amount"] = df["amount"].astype(float)
df["isFraud"] = df["isFraud"].astype(int)
```

4.5 Mapping des colonnes (camelCase → snake_case)

```
column_mapping = {
    "nameOrig" → "name_orig",
    "oldbalanceOrig" → "old_balance_orig",
    "newbalanceOrig" → "new_balance_orig",
    "nameDest" → "name_dest",
    "oldbalanceDest" → "old_balance_dest",
    "newbalanceDest" → "new_balance_dest",
    "isFraud" → "is_fraud",
    "isFlaggedFraud" → "is_flagged_fraud",
}
```

Pourquoi ? En SQL, on utilise snake_case par convention. Les données source utilisent camelCase (JavaScript).

5. ÉTAPE 3 — Chargement PostgreSQL

5.1 Le schéma de la table

```
CREATE TABLE transactions (
    id SERIAL PRIMARY KEY, -- Auto-incrémenté
    transaction_date TIMESTAMP NOT NULL, -- Date+heure
    transaction_date_only DATE NOT NULL, -- Date seule
    transaction_hour INTEGER NOT NULL, -- Heure (0-23)
    step INTEGER NOT NULL, -- Step original
    type VARCHAR(20) NOT NULL, -- PAYMENT, TRANSFER, etc.
    amount NUMERIC(15,2) NOT NULL, -- Montant
    name_orig VARCHAR(20) NOT NULL, -- ID émetteur
    old_balance_orig NUMERIC(15,2), -- Solde avant
    new_balance_orig NUMERIC(15,2), -- Solde après
    name_dest VARCHAR(20) NOT NULL, -- ID destinataire
    old_balance_dest NUMERIC(15,2), -- Solde destinataire avant
    new_balance_dest NUMERIC(15,2), -- Solde destinataire après
    is_fraud INTEGER DEFAULT 0, -- 0=non, 1=oui
    is_flagged_fraud INTEGER DEFAULT 0, -- 0=non, 1=oui
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

5.2 Pourquoi ces types ?

Colonne	Type choisi	Raison
amount	NUMERIC(15,2)	Precise pour l'argent (pas de flottant)
name_orig	VARCHAR(20)	IDs comme "C123456789" (10 caractères)

Colonne	Type choisi	Raison
is_fraud	INTEGER (0/1)	Plus simple que BOOLEAN pour l'analyse
step	INTEGER	Conservation de la valeur originale

5.3 Index créés

```
CREATE INDEX idx_transactions_date ON transactions(transaction_date_only);
CREATE INDEX idx_transactions_type ON transactions(type);
CREATE INDEX idx_transactions_fraud ON transactions(is_fraud);
CREATE INDEX idx_transactions_orig ON transactions(name_orig);
CREATE INDEX idx_transactions_dest ON transactions(name_dest);
```

Pourquoi ? Sans index, PostgreSQL fait un "seq scan" (lit toute la table) pour chaque requête. Avec un index, il trouve directement les lignes.

5.4 Le code de chargement

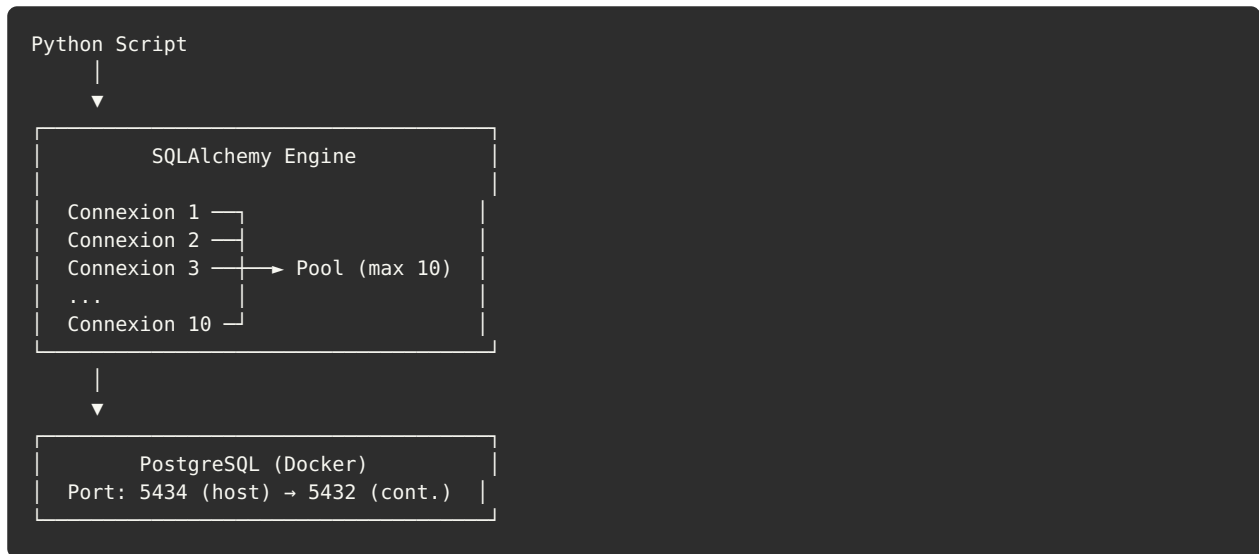
```
# database.py – Mécanisme clé

# 1. Création de l'engine SQLAlchemy
engine = create_engine(
    conn_str,
    pool_pre_ping=True,      # Vérifie si la connexion est vivante
    pool_size=10,            # 10 connexions simultanées
    pool_recycle=300,        # Recycle après 5 minutes
)

# 2. Chargement avec pandas
df.to_sql(
    "transactions",          # Nom de la table
    engine,                  # L'engine SQLAlchemy
    if_exists="append",      # Ajouter (pas écraser)
    index=False,             # Pas d'index dans les données
    chunksize=5000,          # 5000 lignes par batch
)

# 3. Retry en cas d'erreur
for attempt in range(1, max_retries + 1):
    try:
        df.to_sql(...)
        return len(df)
    except Exception as e:
        if attempt < max_retries:
            time.sleep(attempt * 5) # Backoff exponentiel
            engine.dispose()        # Nettoyer le pool
```

5.5 Le pool de connexions



Pourquoi un pool ? - Ouvrir/fermer une connexion est coûteux - Le pool réutilise les connexions -
`pool_pre_ping` détecte les connexions mortes - `pool_recycle` évite les timeouts

6. Docker — PostgreSQL

6.1 Le docker-compose.yml

```
services:
  postgres:
    image: postgres:16 # Image officielle
    container_name: transaction_postgres
    restart: unless-stopped # Redémarrage auto
    ports:
      - "5434:5432" # Host:5434 -> Container:5432
    environment:
      POSTGRES_DB: ${DB_NAME:-transaction_db}
      POSTGRES_USER: ${DB_USER:-kevin}
      POSTGRES_PASSWORD: ${DB_PASSWORD:-kevin123}
    volumes:
      - pgdata:/var/lib/postgresql/data # Données persistantes
      - ./sql:/docker-entrypoint-initdb.d # Init auto
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ${DB_USER:-kevin}"]
      interval: 5s
      timeout: 5s
      retries: 5
```

6.2 Pourquoi Docker ?

Sans Docker	Avec Docker
Installer PostgreSQL manuellement	<code>docker-compose up -d</code>
Configurer les ports	Port mapping automatique
Gérer les mises à jour	<code>docker pull postgres:16</code>
Risque de conflit	Isolation complète

6.3 Le volume `pgdata`

```
Docker host
├─ /var/lib/docker/volumes/pgdata/_data/
│   ├── base/          ← Données PostgreSQL
│   ├── pg_wal/        ← Write-Ahead Log
│   └── ...
```

Important : Les données survivent au redémarrage du container. Pour tout supprimer : `docker-compose down -v`.

7. L'environnement Virtuel (venv)

7.1 Pourquoi un venv ?

Sans venv :	Avec venv :
Python système	venv du projet
├─ pandas 1.x (ancien)	├─ pandas 3.0.5
├─ sqlalchemy 1.4	├─ sqlalchemy 2.0.52
└─ psycopg2 2.8	└─ psycopg2 2.9.12
Problème : conflits	Solution : isolation

7.2 Création et utilisation

```
# Créer le venv
python3 -m venv venv

# Activer le venv (OBLIGATOIRE avant de lancer)
source venv/bin/activate

# Installer les dépendances
pip install -r requirements.txt

# Vérifier
python -c "import pandas; print(pandas.__version__)"
```

7.3 Pourquoi "source venv/bin/activate" ?

Le script `activate` modifie temporairement : - La variable `PATH` pour utiliser le Python du venv - Le prompt shell pour afficher `(venv)` - Les variables d'environnement

Sans activation, Python utilise les packages du système (qui n'ont pas les bonnes versions).

8. Le fichier `.env`

8.1 Contenu

```
DB_HOST=localhost
DB_PORT=5434
DB_NAME=transaction_db
DB_USER=kevin
DB_PASSWORD=kevin123
```

8.2 Pourquoi .env ?

- **Sécurité** : Les credentials ne sont pas dans le code
- **Flexibilité** : Chaque environnement a ses propres credentials
- **Git** : `.env` est dans `.gitignore` (pas versionné)

8.3 Utilisation dans le code

```
from dotenv import load_dotenv
import os

load_dotenv() # Charge le fichier .env

DB_HOST = os.getenv("DB_HOST", "localhost") # Valeur par défaut
DB_PORT = os.getenv("DB_PORT", "5432")
```

9. Les concepts clés à retenir

9.1 Staging vs Transformé

STAGING (données brutes)	TRANSFORMÉ (business-ready)
├─ step = 1	├─ transaction_date = 2024-01-01 00:00:00
├─ nameOrig = "C123456789"	├─ name_orig = "C123456789"
├─ isFraud = 0	├─ is_fraud = 0
└─ Pas de timestamp	├─ transaction_date_only = 2024-01-01
	└─ transaction_hour = 0

Règle : On ne modifie jamais les données staging. On crée une nouvelle version transformée.

9.2 Idempotence

Un pipeline est **idempotent** si rejouer la même opération produit le même résultat.

```
# PAS idempotent : si on rejoue, on duplique les données
df.to_sql("transactions", engine, if_exists="append")

# Idempotent : on écrase les données existantes
df.to_sql("transactions", engine, if_exists="replace")
```

Dans notre cas, on utilise `append` + `DROP TABLE IF EXISTS` au début. C'est idempotent car on recrée la table à chaque fois.

9.3 Pool de connexions

```
Requête 1 → Connexion 1 ┌─┐
Requête 2 → Connexion 2 ┌─┐ → Pool → PostgreSQL
Requête 3 → Connexion 1 └─┘ (réutilise)
```

Avantage : Pas besoin d'ouvrir/fermer une connexion pour chaque requête.

9.4 Retry avec backoff

```
# Tentative 1 : échec → attendre 5s
# Tentative 2 : échec → attendre 10s
# Tentative 3 : échec → abandonner
```

Pourquoi ? Si PostgreSQL est temporairement surchargé, attendre puis réessayer est plus intelligent que crasher immédiatement.

10. Commandes essentielles

10.1 Démarrer le projet

```
# 1. Cloner
git clone <url>
cd Transaction_numerique

# 2. Environnement
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt

# 3. PostgreSQL
docker-compose up -d

# 4. Pipeline
python -m src.ingestion.ingest_batch
python -m src.transformation.transform_transactions
python -m src.utils.database

# 5. Tests
pytest tests/ -v
```

10.2 Commandes Docker

```
docker-compose up -d      # Démarrer
docker-compose down       # Arrêter
docker-compose down -v    # Arrêter + supprimer les données
docker-compose logs postgres # Voir les logs
docker ps                 # Vérifier les containers
```

10.3 Commandes PostgreSQL

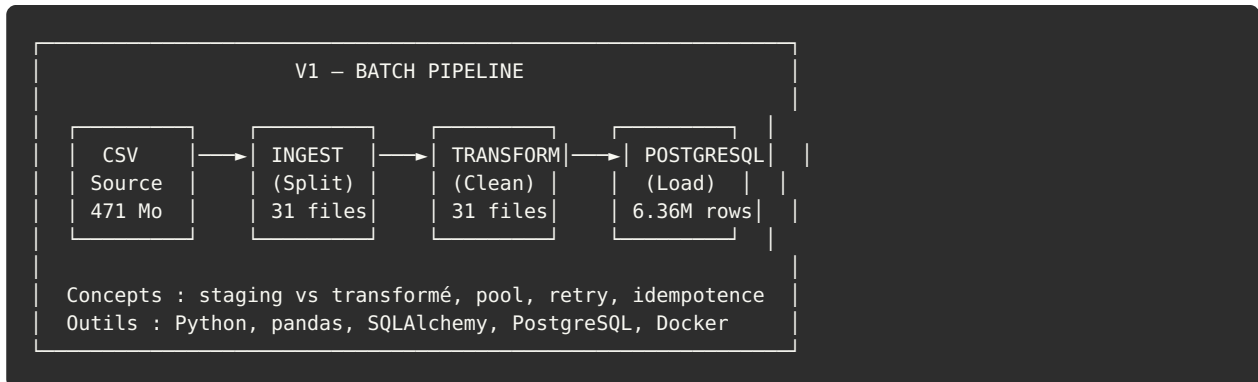
```
psql -h localhost -p 5434 -U kevin -d transaction_db
# SQL :
SELECT COUNT(*) FROM transactions;
SELECT type, COUNT(*) FROM transactions GROUP BY type;
```

11. Dépannage

Problème	Cause	Solution
ModuleNotFoundError	venv pas activé	source venv/bin/activate
Connection refused	PostgreSQL pas démarré	docker-compose up -d
FATAL: password authentication failed	Mauvais credentials	Vérifier .env
server closed the connection	Connexion stale	pool_pre_ping=True + retry
timeout expired	Requête trop longue	Augmenter chunksize

Problème	Cause	Solution
database is locked	Autre process utilise la DB	Arrêter l'autre process

12. Résumé en une image



Document généré pour le projet Transaction Data Platform — V1 Batch